

CS540 Fall 2026 Homework 2

Due: Friday, September 25, 2026, 11:59 PM (CST)

1 Probabilistic Language Identification

Your detectives arrived at company X only to find an important piece of evidence, a business letter, has been shredded. After painstakingly recovering the pieces, the detectives reported to you that all they could do was count how many times each alphabet character appeared in the letter. Your mission, should you choose to accept it, is to estimate whether the letter was written in English or Spanish. The letter is known to be entirely written in only one language.

You are given a python program skeleton **hw2.py**, which you are to complete. Your program should read from 3 input files to hw2.py (cf. **section 1.3 for more details**), one of which is a text file we will supply when we grade your program. This text file will be processed by a “digital shredder,” provided to you in the python program skeleton, which reads in a plain text letter and stores the character counts in a dict (ignoring case; known as case-folding). Any one of the *samples/letter[0-4].txt* files can be used to test your code in conjunction with the “digital shredder.”

Reminder: you are allowed to discuss broad ideas with each other and the teaching team (instructors, TAs, and PMs), but you should disclose broad discussion points in comments.

Use of AI tools and other resources on homework is allowed: you may use artificial intelligence tools (such as ChatGPT, Claude, Gemini, or Cursor) as well as books or online resources on homework without restriction. Note that the exams are closed-book and worth 70% of your grade, so the homework is your main opportunity to learn the material.

1.1 Language identification via Bayes rule

We arrange the 26 counts into a 26-dimensional count vector

$$X = (X_1, \dots, X_{26}),$$

where X_1 is the count of A, X_{26} is the count of Z, and so on. X is the observed evidence. We are interested in the source language $y \in \{English, Spanish\}$. Our goal is to compute the conditional probability

$$P(Y = y | X).$$

For this, we will use the Bayes rule:

$$P(Y = y | X) = \frac{P(X | Y = y)P(Y = y)}{P(X)} = \frac{P(X | Y = y)P(Y = y)}{\sum_{y' \in \{English, Spanish\}} P(X | Y = y')P(Y = y')}.$$

We need to know the following:

- $P(Y = y)$ the prior probability that the letter is of language y , without seeing any shredded evidence. **The hw2.py code takes as input the two priors; see the command in section 1.3.** For local tests for this homework, use the following priors, but *do not* hardcode the values into hw2.py:

$$P(Y = English) = 0.58, P(Y = Spanish) = 1 - P(Y = English) = 0.42.$$

- $P(X | Y = y)$ the conditional probability of observing evidence X given language y . We will do this using the multinomial probability. Under this model, for $y = English$ let

$$e = (e_1, \dots, e_{26})$$

be a 26-dimensional multinomial parameter vector of English where $e_i \geq 0$ for $i = 1 \dots 26$, and $\sum_{i=1}^{26} e_i = 1$. e_i is the probability of the i th character in English, i.e. e_1 is for A, e_{26} is for Z. Similarly, let

$$s = (s_1, \dots, s_{26})$$

be the parameter vector of Spanish. e and s are given to you as two plain text files **e.txt**, **s.txt** with self-explanatory format.

Then, the multinomial probability is defined as

$$P(X | Y = y) = C(X) \prod_{i=1}^{26} p_i^{X_i}$$

where p is e if $y = English$ or s if $y = Spanish$, and $C(X)$ is the multinomial coefficient:

$$C(X) = \frac{\left(\sum_{i=1}^{26} X_i\right)!}{X_1! \cdot \dots \cdot X_{26}!}.$$

You now have everything needed to mathematically compute $P(Y = y | X)$ for $y \in \{English, Spanish\}$.

1.2 Computational considerations

However, as a computer scientist we want to avoid unnecessary computation and to avoid underflow. By inspecting $P(Y = y \mid X)$ and remembering the normalization $P(Y = \textit{English} \mid X) + P(Y = \textit{Spanish} \mid X) = 1$, we see that any terms that do not depend on y can be ignored, as long as we normalize in the end. Concretely, we say

$$P(Y = y \mid X) \propto P(Y = y) \prod_{i=1}^{26} p_i^{X_i}$$

where we removed $C(X)$ and $P(X)$. Equivalently, we can compute (recall p depends on y)

$$f(y) = P(Y = y) \prod_{i=1}^{26} p_i^{X_i}$$

and then normalize:

$$P(Y = y \mid X) = \frac{f(y)}{f(\textit{English}) + f(\textit{Spanish})}.$$

We also do not like multiplying many small numbers for fear of underflow, so we want to compute things in the log domain:

$$F(y) = \log f(y) = \log P(Y = y) + \sum_{i=1}^{26} X_i \log p_i.$$

We will use the default base in python which is natural log. Also note that, even though the official test cases will never require evaluating $\log(0)$, if your own debugging inputs produce a zero probability, you may handle it by treating $\log(0)$ as a sufficiently large negative value (e.g., -10^8). It is acceptable for debugging only and will not affect grading. In addition, when an expression of the form $0 \log 0$ arises, please use the standard convention $0 \log 0 = 0$.

From $F(y)$ we can compute the desired conditional probability by

$$P(Y = y \mid X) = \frac{e^{F(y)}}{e^{F(\textit{English})} + e^{F(\textit{Spanish})}}.$$

However, $F(\cdot)$ can be very negative if the letter is long. This leads to $e^{F(\cdot)}$ underflow, and we may have a divide-by-zero error that must be handled. To make the computation more robust, let's fix $y = \textit{English}$ and note that

$$P(Y = \textit{English} \mid X) = \frac{e^{F(\textit{English})}}{e^{F(\textit{English})} + e^{F(\textit{Spanish})}} = \frac{1}{1 + e^{F(\textit{Spanish}) - F(\textit{English})}}.$$

Now, $F(\textit{Spanish}) - F(\textit{English})$ can be very positively large and $e^{F(\textit{Spanish}) - F(\textit{English})}$ overflows toward infinity. But we know the ratio should approximate 0 in

that case. Conversely, $F(\text{Spanish}) - F(\text{English})$ can be very negative and $e^{F(\text{Spanish}) - F(\text{English})}$ underflows toward zero. But we know the ratio should approximate 1 in that case. Thus we require that you do an if-else check as follows:

- If $F(\text{Spanish}) - F(\text{English}) \geq 100$ then simply set $P(Y = \text{English} | X) = 0$;
- If $F(\text{Spanish}) - F(\text{English}) \leq -100$ then simply set $P(Y = \text{English} | X) = 1$;
- Otherwise compute

$$P(Y = \text{English} | X) = \frac{1}{1 + e^{F(\text{Spanish}) - F(\text{English})}}.$$

1.3 Program output specification

Your program must handle dynamic input from the command line and produce output that strictly follows the format below for the autograder.

We will run your program using the following command:

```
python3 hw2.py [letter_file] [english_prior] [spanish_prior]
```

Hint: you should be able to access the input parameters with `sys.argv`. The `english_prior` and `spanish_prior` are the probabilities $P(Y = \text{English})$ and $P(Y = \text{Spanish})$, respectively.

Your program should print the output that follows the specification in the first 3 questions. **Do not** print any extra blank lines or debug text.

Q1 ([25] points) Log-probability of the First Letter: Compute $X_1 \log e_1$ and $X_1 \log s_1$. Print “Q1” then these values up to **4 decimal places** on two separate lines. Sample output for `letter1.txt` is as follows:

```
Q1
-0.0000
-0.0000
```

Output format explanation: The first line is the value of $X_1 \log e_1$ and the second line is the value of $X_1 \log s_1$. There is no empty line in between or additional blank spaces. Notice that the values are 0 since 'A' (or 'a') does not appear in the text! The negative sign is due to python implementation, we also accept 0.0000 as an answer.

Q2 ([25] points) Total Log-probability: Compute $F(\text{English})$ and $F(\text{Spanish})$. Similarly, print “Q2” followed by their values up to **4 decimal places** on two separate lines.

Q3 ([30] points) Posterior Probability: Compute $P(Y = \text{English} | X)$.

Print “Q3” then this value up to **4 decimal places**.

You are expected to print out the answers to the above questions to stdout. In other words, your program should print the values to your screen. You can use the python **print()** function. Do not add additional blank lines between your outputs. Carefully follow the format described above as your submission will be automatically graded using an autograder. We provided five sample input and output files for your reference.

Q4 ([20] points) Prior Sensitivity Experiment: In this experiment, you will investigate how the prior probabilities affect the posterior probability while the observed letter remains fixed. Use **letter0.txt** for all three runs below:

```
python3 hw2.py samples/letter0.txt 0.50 0.50
python3 hw2.py samples/letter0.txt 0.90 0.10
python3 hw2.py samples/letter0.txt 0.10 0.90
```

For each run, record the value of $P(Y = \textit{English} \mid X)$ reported in Q3. Create a plain text file named **prior_sensitivity.txt** with the following format:

```
English prior: 0.50, Spanish prior: 0.50, Posterior: [value]
English prior: 0.90, Spanish prior: 0.10, Posterior: [value]
English prior: 0.10, Spanish prior: 0.90, Posterior: [value]
```

- A. [your answer]
- B. [your answer]
- C. [your answer]

Replace each [value] with the corresponding posterior probability up to **4 decimal places**, and answer the following questions in one or two sentences each:

- A. Which quantities in the computation remain unchanged across the three runs, and which quantities change?
- B. Does changing the prior change the predicted language in any of the three runs? Explain your answer using the relative strength of the prior and the evidence from the observed letter.
- C. For a much longer letter generated from one of the two language models, would you generally expect the prior to have more or less influence on the final prediction? Explain briefly.

Q4 does not change the required standard output format of Q1–Q3. Do not print the contents of **prior_sensitivity.txt** from **hw2.py**.

1.4 Files in the assignment

The homework is released as a single **hw2.zip** which contains the following files and directories:

- English character probability file **e.txt**
- Spanish character probability file **s.txt**
- Directory **samples** contains sample letters and the desired output and the python file **compare_hw2.py** used for comparison. Your outputs should be identical to the ones provided. One way to check whether your output matches the expected output is to use the provided **compare_hw2.py** on Linux Terminals (use CSL Machines) for a line by line comparison between two files. Usage:

```
>> python3 compare_hw2.py <your_output.txt> <letterX_out.txt>
```

Use **letter0.txt** as an example, you can save your output to a file and then compare as follows

```
>> python3 hw2.py samples/letter0.txt 0.58 0.42 | tee your_output.txt
>> python3 compare_hw2.py your_output.txt samples/letter0_out.txt
```

- Skeleton code **hw2.py** to help you get started. It contains the following two functions:
 - **get_parameter_vectors()**: Reads "e.txt" and "s.txt" and returns the e and s vectors described in section 1.1. We have already implemented this function for you. You can directly use the returned values in your code.
 - **shred()**: Implements digital shredder functionality, which reads a text file and returns the character counts in a dict (ignoring case), as described in the header of the write-up. We have already implemented this function for you. You can directly use the returned values in your code.
- Template file **prior_sensitivity.txt** for recording the three Q4 posterior probabilities and the written responses A–C.

Take special note of the contents of **letter4.txt**. This is a letter written in German. What does our model predict (English or Spanish)? Our model should only know about English and Spanish, but why is the model so confident even though the input is neither English nor Spanish? It is useful to think about how the model will respond to languages it doesn't know.

You are free to implement the rest of the assignment as you wish using your python coding skills.

1.5 Deliverables

Please submit your files named **hw2.py** and **prior_sensitivity.txt** to Gradescope. **Do not** submit a Jupyter notebook (.ipynb) file. Make sure you use **Python3** and **only** use the Python **built-in standard library** (do not use third-party packages like numpy or pandas).

To check whether a library is built-in (for Python 3.10+), you can run:

```
import sys
# Replace "NAME_OF_LIBRARY" with the library you want to check
print("NAME_OF_LIBRARY" in sys.stdlib_module_names)
```

ALL THE BEST!